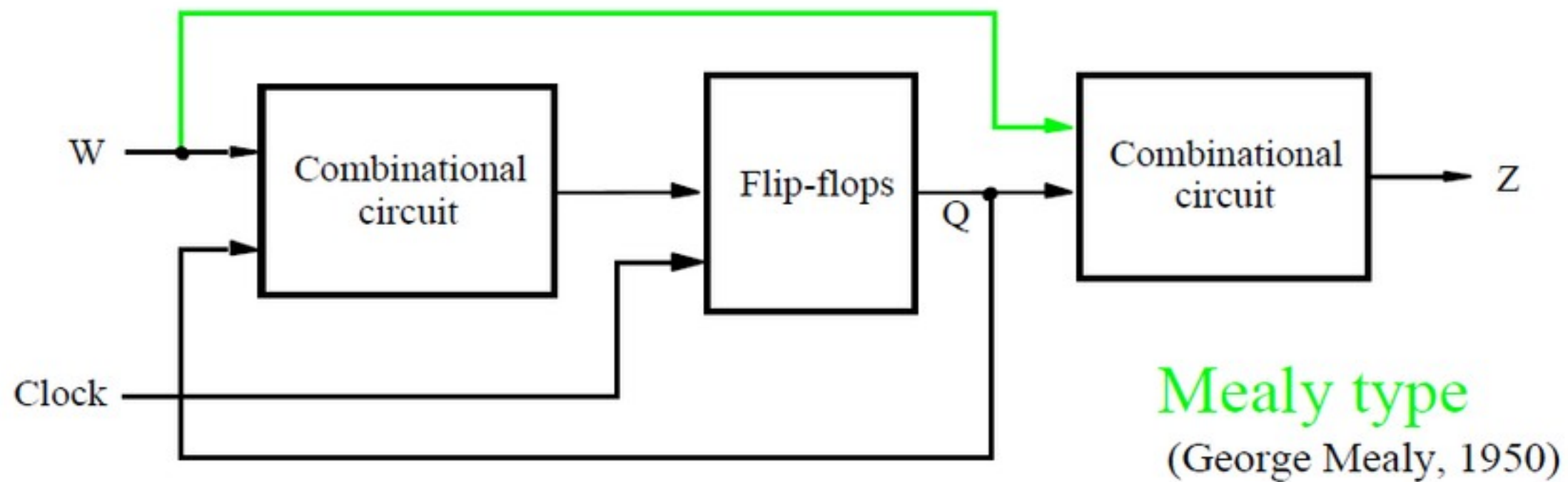
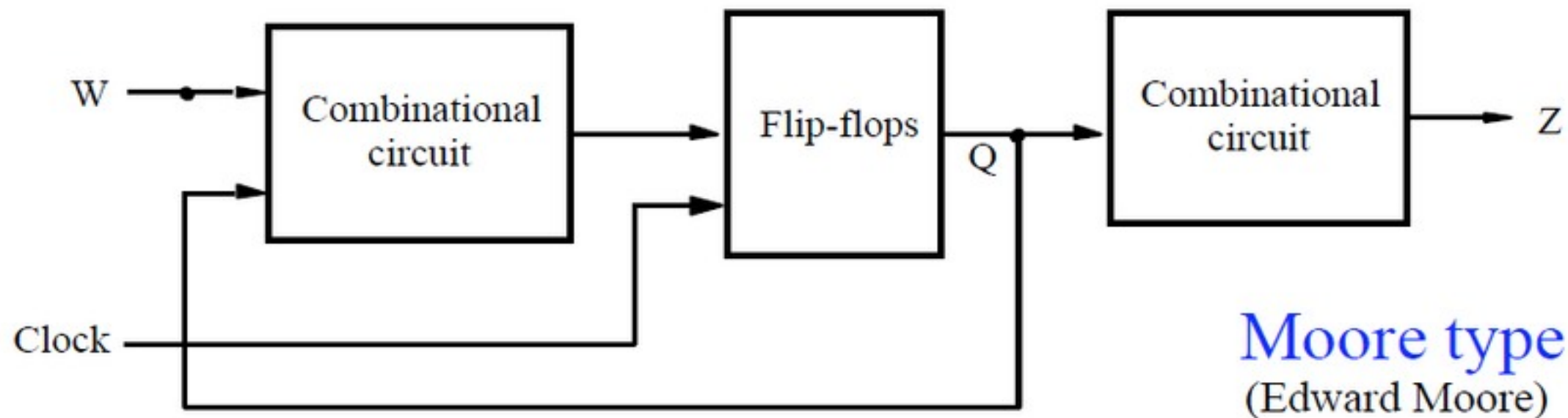


19ECE204
Digital Electronics and Systems
III Semester
B. Tech. (ECE & CSE)
Amrita School of Engineering, Bengaluru
Synchronous Sequential Circuits

Synchronous Sequential Circuits



Sequential Circuits

We considered combinatorial logic circuits in which outputs are fully determined by the present values of the inputs.

We also considered simple storage elements that can be implemented in the form of flipflops for which the output depends on their states rather than the value of the inputs; the inputs cause changes in the state.

In this lecture we deal with a general class of circuits in which the output depend on the past behavior of the circuit, as well on the present values of the inputs.

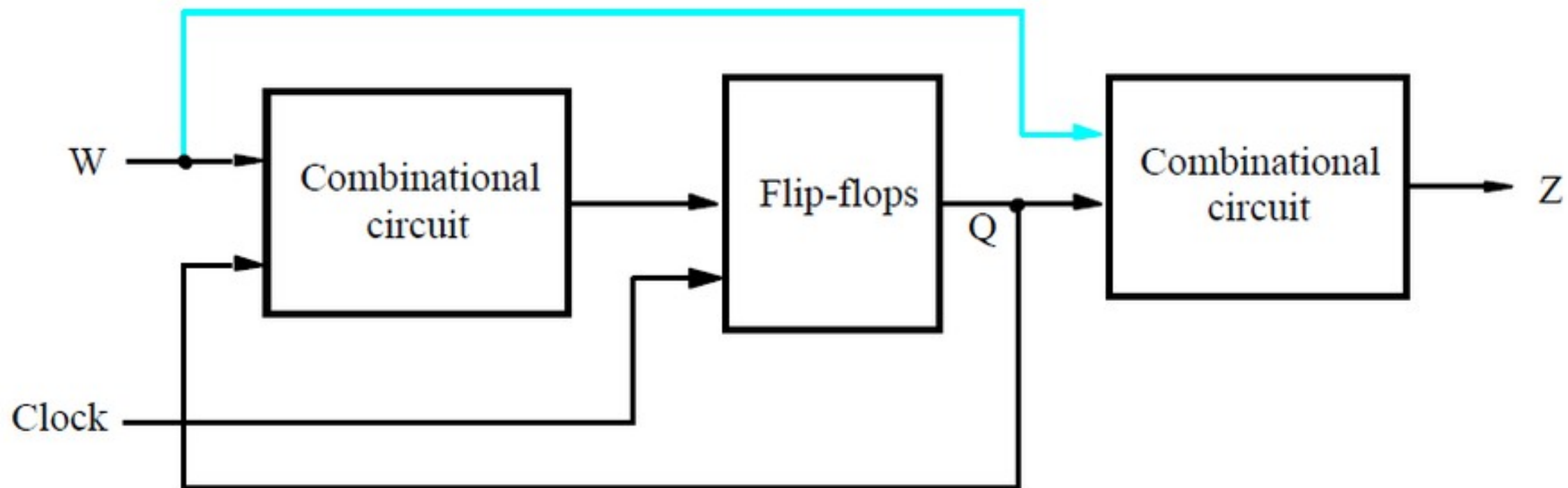
They are called *sequential circuits*.

Synchronous vs Asynchronous

In most cases a clock signal is used to control the operation of a sequential circuit. Such circuit is called *synchronous*.

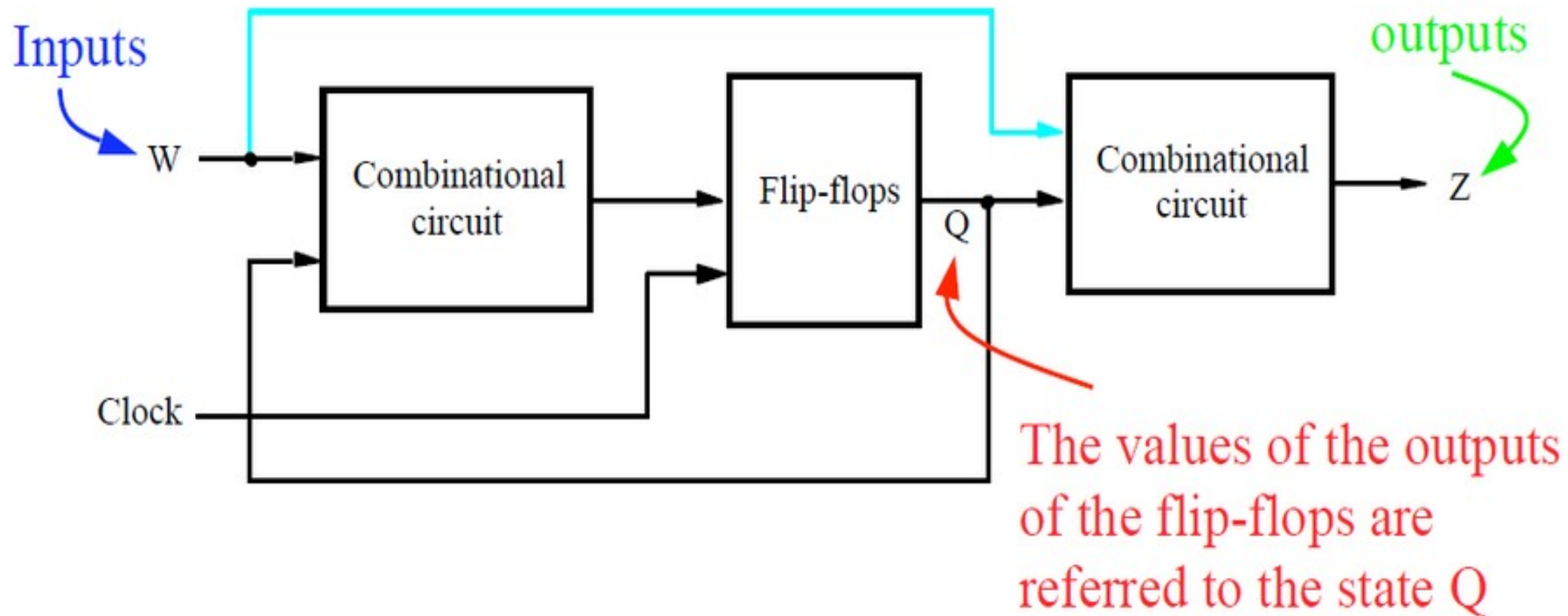
- The alternative, in which clock signal is not used, is called *asynchronous sequential circuits*.
- Synchronous circuits are easier to design and are used in the vast majority of practical applications.

Synchronous Sequential Circuits



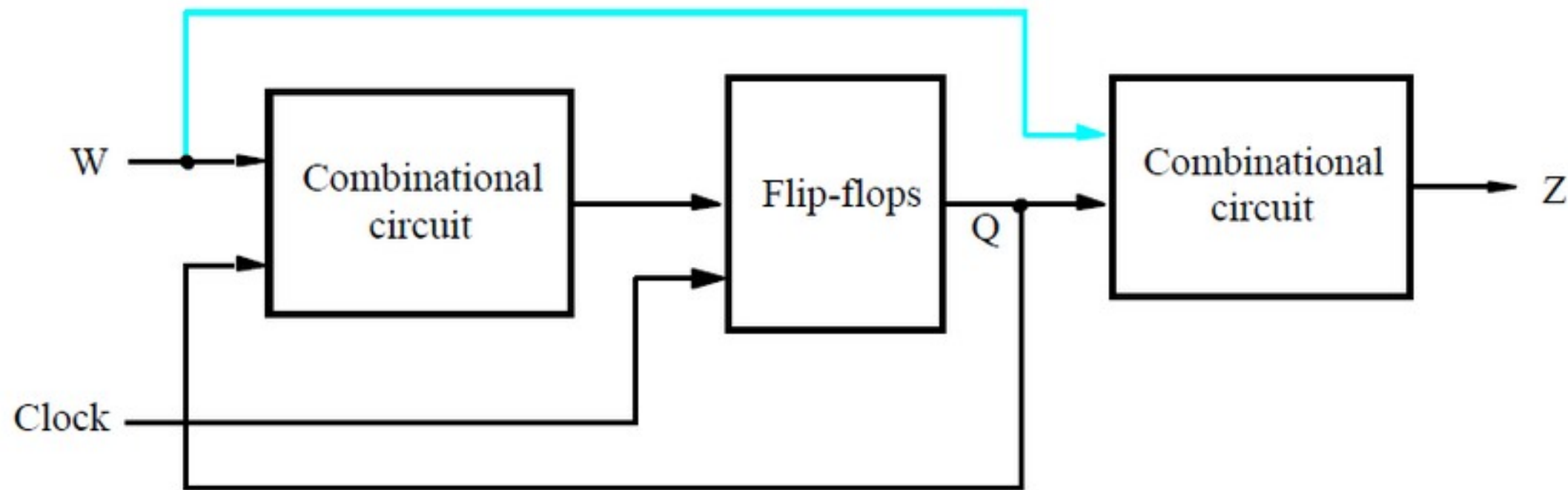
- This circuits are realized with combinatorial logic and one or more flip-flops.

Synchronous Sequential Circuits



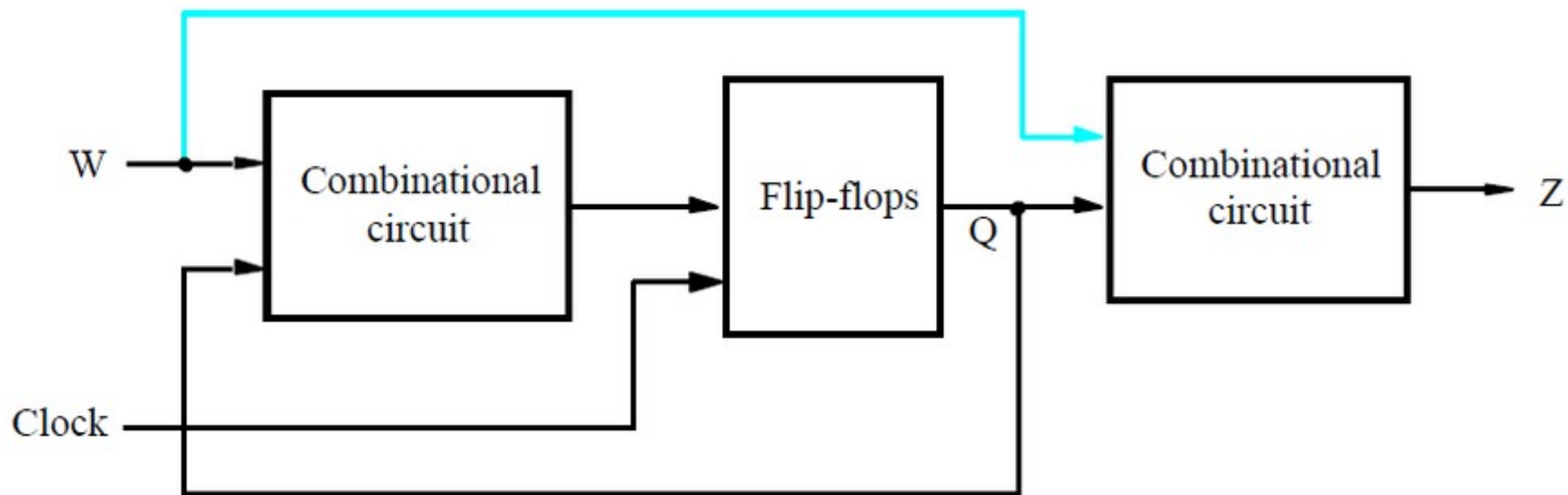
Under the control of the clock signal, the flip-flop outputs change their state as determined by the combinatorial logic that feeds the inputs of the flip-flops. Therefore the circuit moves from one state to the next: $Q_t \rightarrow Q_{t+1}$.

Synchronous Sequential Circuits



- The combinatorial logic that provide the input signals of the flip flops derives its inputs from two sources: the primary inputs W , and the current outputs of the flip-flops Q . Thus changes in state depend on both present state and the values of the primary inputs.
- The outputs of the circuit are generated by another combinatorial circuit, such that the outputs are a function of the present state and the primary inputs.

Synchronous Sequential Circuits



- Although the outputs always depend on the present state, they don't have to depend directly on the primary inputs. Thus the connection in blue may not exist.

Finite State Machines (FSM)

- FSM = Synchronous Sequential Circuits
- The name derives from the fact that the functional behavior of these circuits can be represented using a finite number of states.





Basic Design Steps (1)

- We introduce the techniques for designing sequential circuits by means of a simple example.
- Suppose that we want to design a circuit that meets the following specifications:
 - The circuit has an input and an output
 - All changes in the circuit occur on the positive edge of a clock signal
 - The output z is equal to 1 if during two immediately preceding clock cycles the input w equal to 1. Otherwise the value of z is set to 0.
- Thus, the circuit detects if two or more cosecutive 1s occur on its input w .
- Circuits that detect the occurrence of a particular pattern on its input(s) are referred as *sequence detectors*.

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
$w:$	0	1	0	1	1	0	1	1	1	0	1
$z:$	0	0	0	0	0	1	0	0	1	1	0

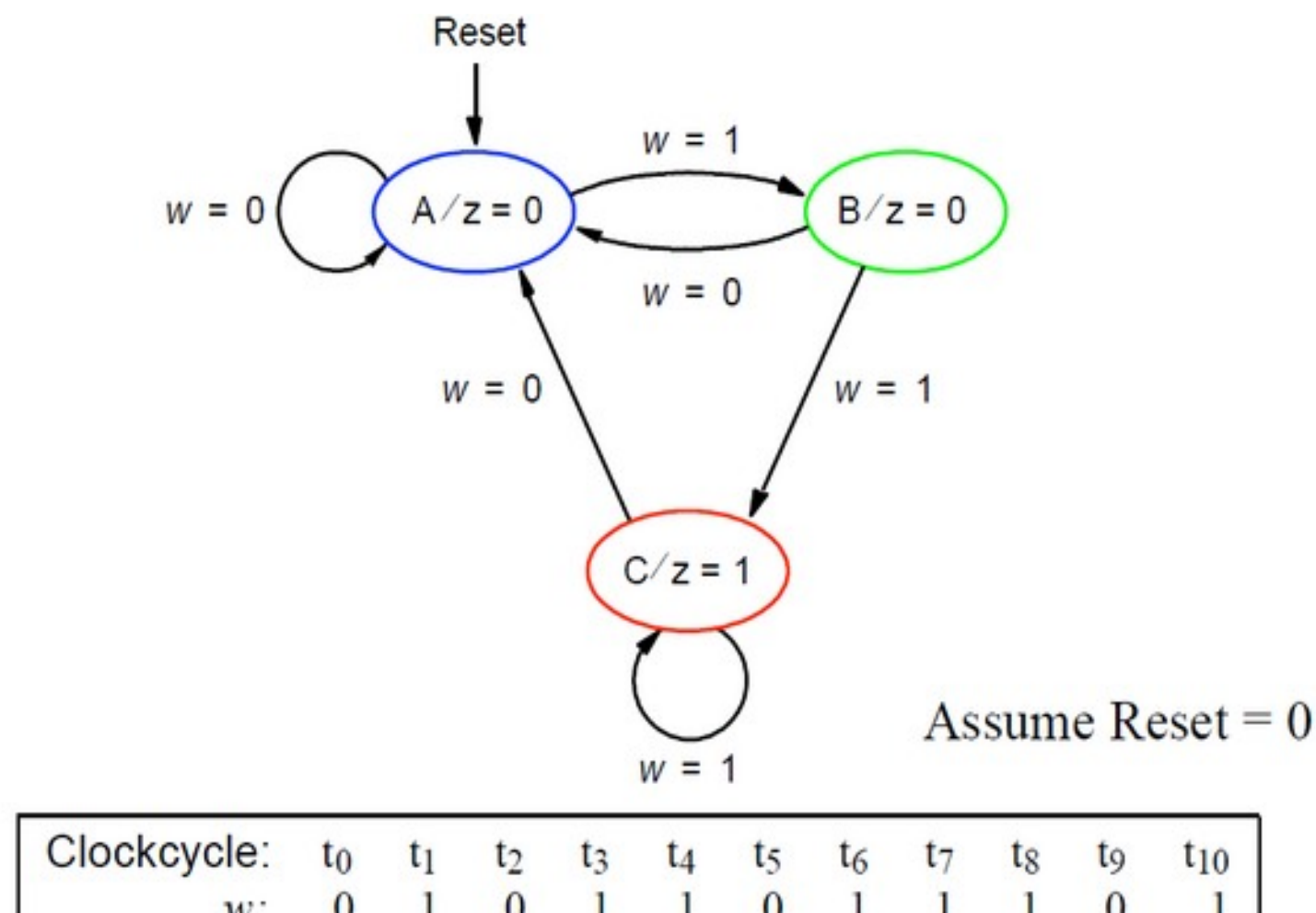
Sequences of input and output signals.

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	0	1	0	0	1	1	0



State Diagram

- The conceptually simplest method to represent the behavior of a FSM is to use a pictorial representation in the form of a state diagram, which is a graph that depicts the state of the circuit as nodes (circles) and transitions between states as arcs.



State Table

- State diagrams provide a description of the behavior of a sequential circuit that is easy to understand.
- To proceed with the implementation of the circuit it is convenient to translate the information contained in the diagram in tabular form.

State Table

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

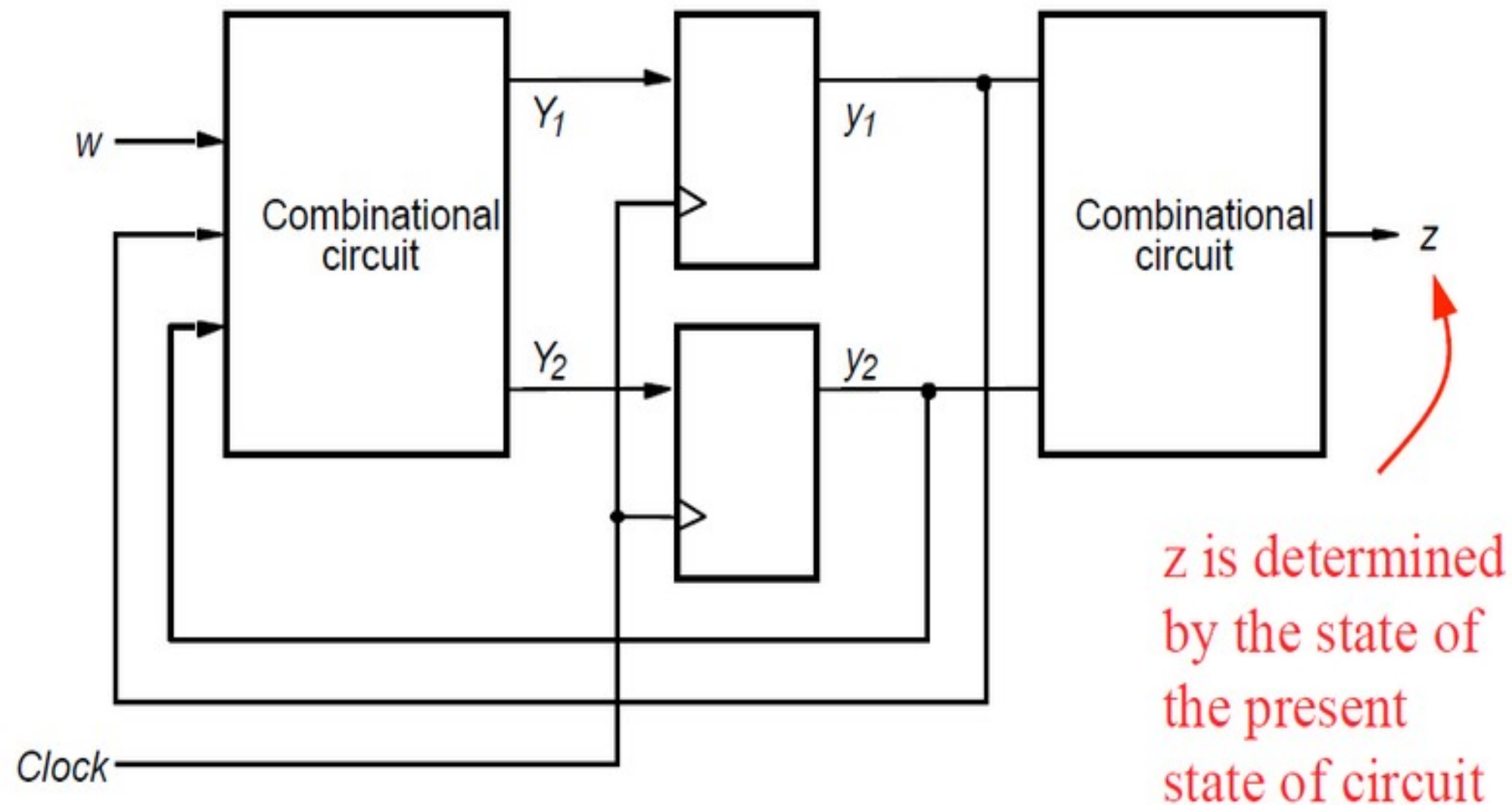
- The table indicate all transitions from each present state to the next state for different values of the input signal.
- The output signal is defined with respect to the present state.
- We did not include the Reset input signal; instead we made the implicit assumption that the first state in the table is the starting state.

State Assignment(1)

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

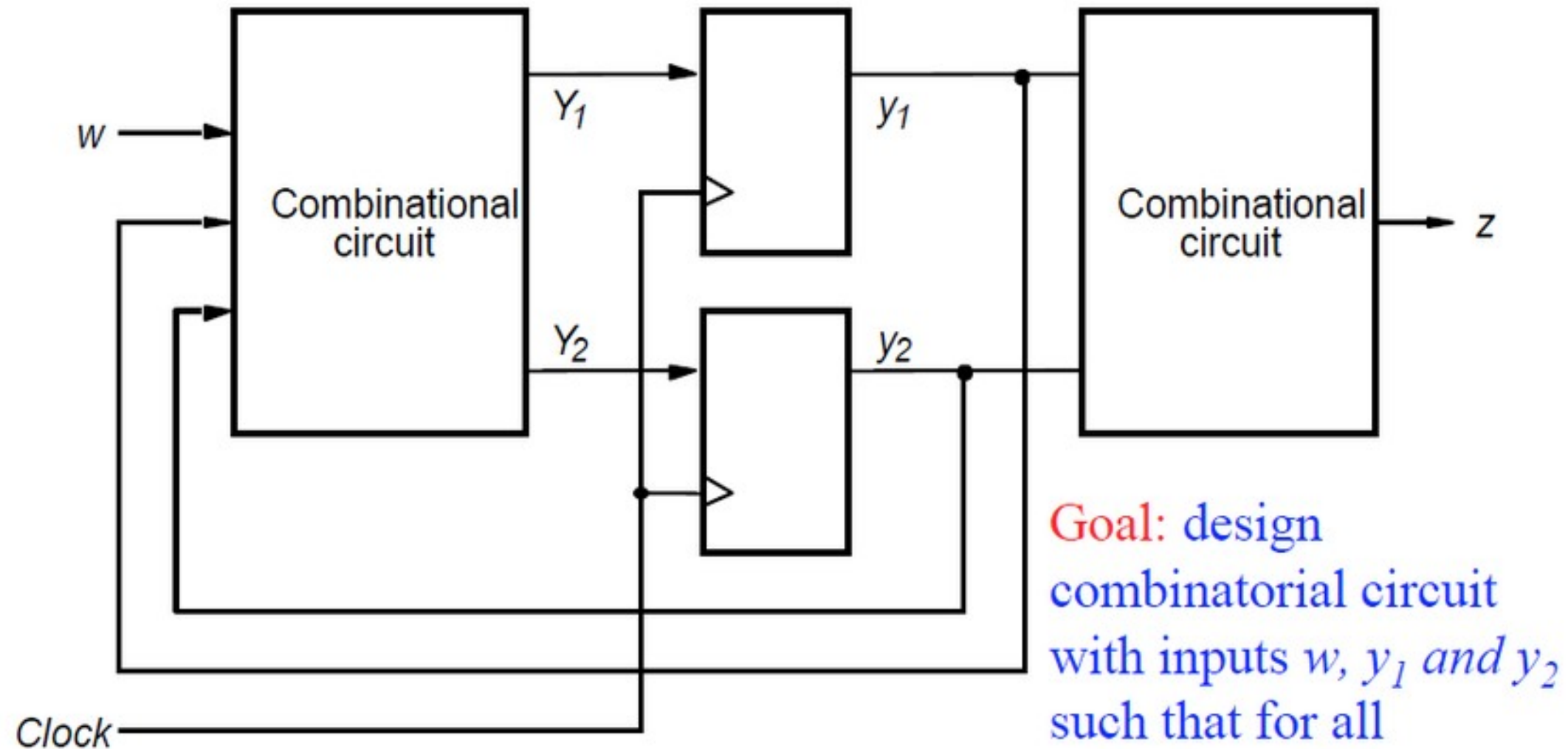
- When implemented as a logic circuit, each state is represented by the valuation (combination of values) of *state variables*.
- Each state may be implemented in the form of a flip-flop.
- Since three states have to be realized, it is sufficient to use two state variables.

State Assignment (2)



- The figure shows the structure of the circuit that implements the FSM of the problem (Moore type).

State Assignment (3)



- $Z = f(y_1, y_2)$.
- y_1, y_2 - present-state variables.
- Y_1, Y_2 - next-state variables.

Goal: design combinatorial circuit with inputs w, y_1 and y_2 such that for all valuations of these inputs the outputs Y_1, Y_2 will cause the FSM to move to the next state that satisfies our specs.

State Assignment(4)

Present state	Next state		Output z
	$w = 0$	$w = 1$	
A	A	B	0
B	A	C	0
C	A	C	1

- When implemented as a logic circuit, each state is represented by the valuation (combination of values) of *state variables*.
- Each state may be implemented in the form of a flip-flop.
- Since three states have to be realized, it is sufficient to use two state variables.

	Present state y_2y_1	Next state		Output z
		$w = 0$	$w = 1$	
		Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

$$Y_1 = D_1 = w$$

$$Y_2 = D_2 = wy_1$$

$$z = y_2$$





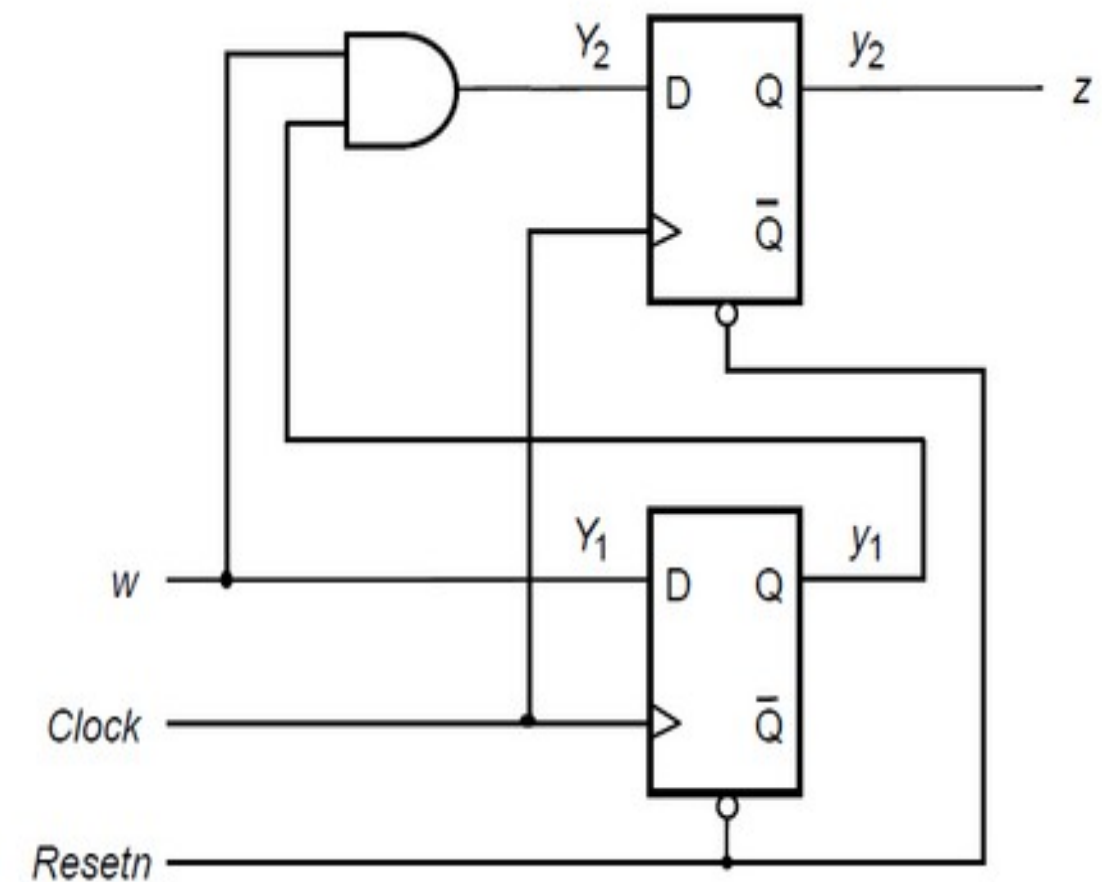


	Present state y_2y_1	Next state		Output z
		$w = 0$	$w = 1$	
		Y_2Y_1	Y_2Y_1	
A	00	00	01	0
B	01	00	11	0
C	11	00	11	1
	10	dd	dd	d

$$Y_1 = D_1 = w$$

$$Y_2 = D_2 = wy_1$$

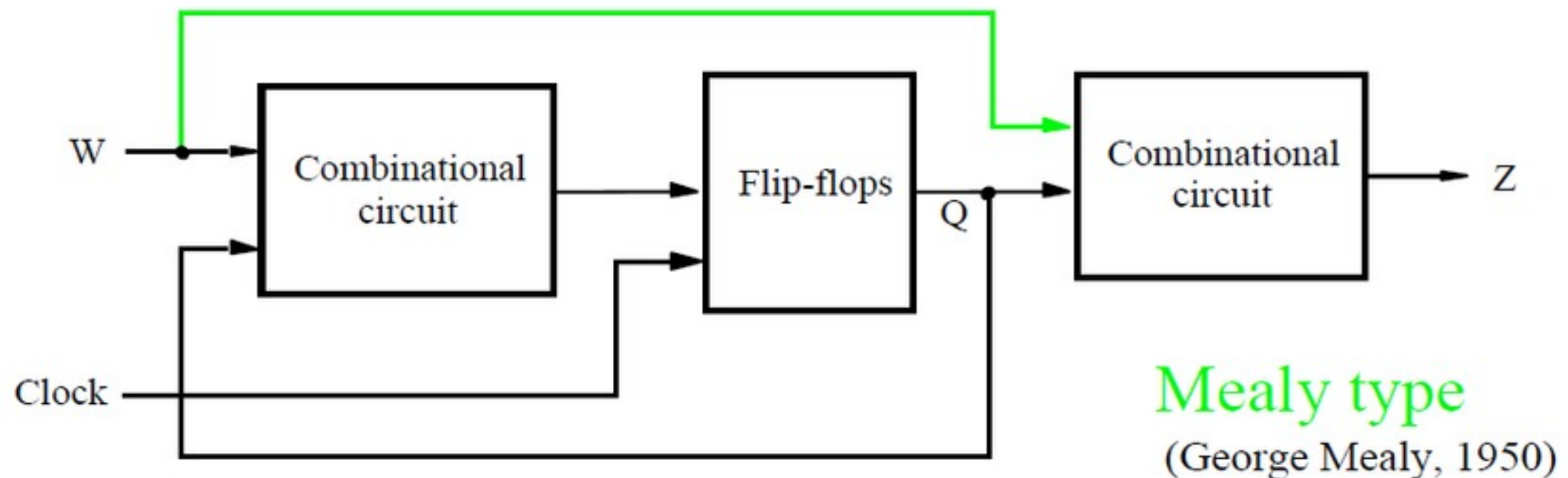
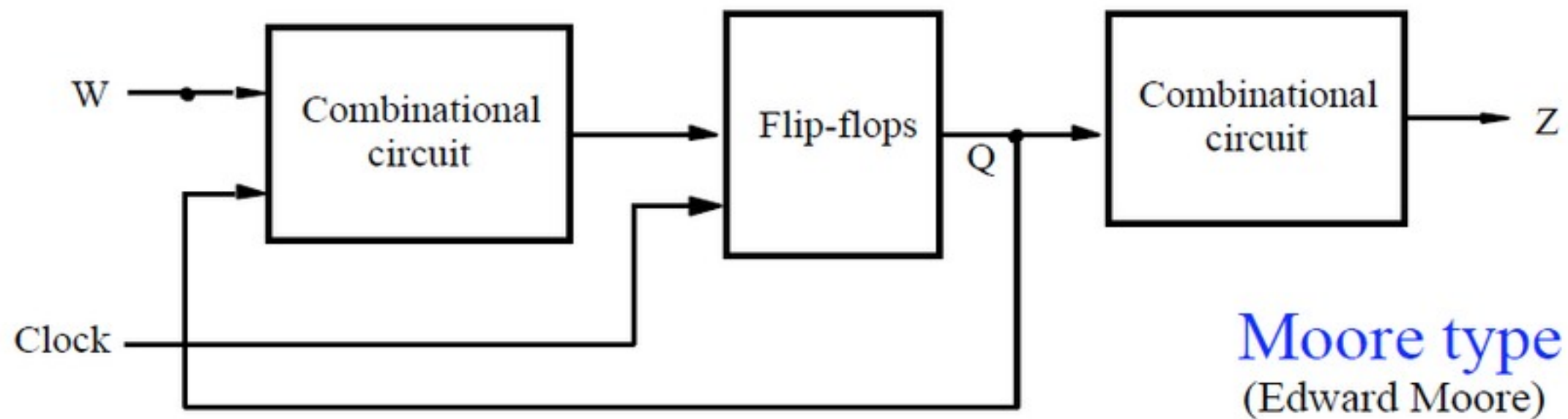
$$z = y_2$$







Synchronous Sequential Circuits



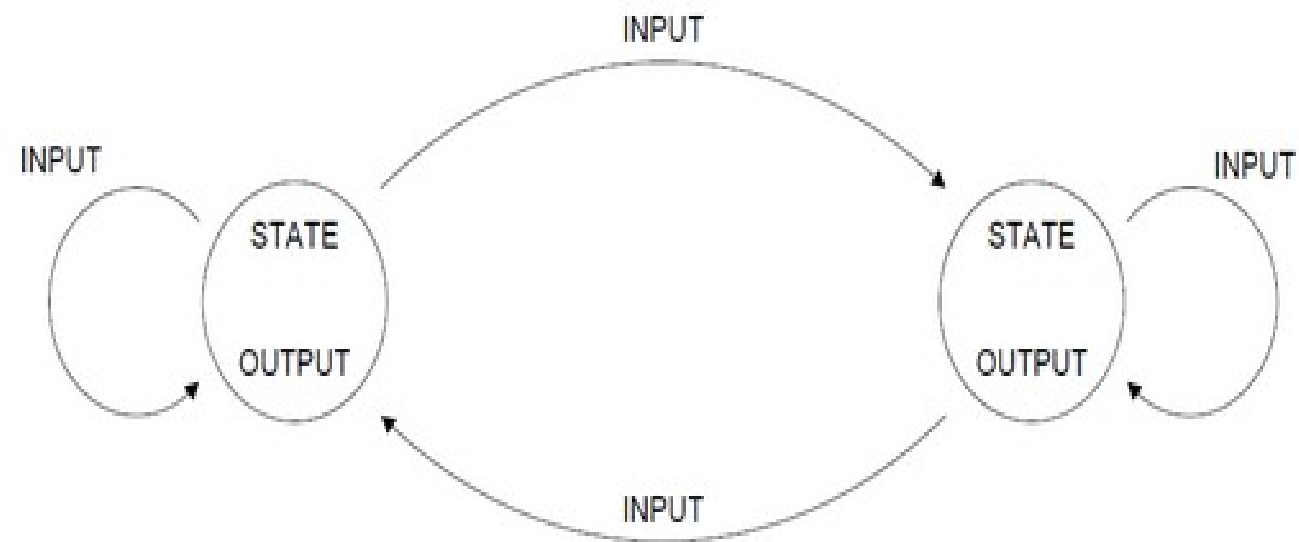
Finite State Machines (FSM)

- FSM = Synchronous Sequential Circuits
- The name derives from the fact that the functional behavior of these circuits can be represented using a finite number of states.



Moore State Machines:

- Outputs determined **solely** by the current state
- Outputs are *unconditional* (not directly dependent on input signals)



GENERIC MOORE STATE MACHINE

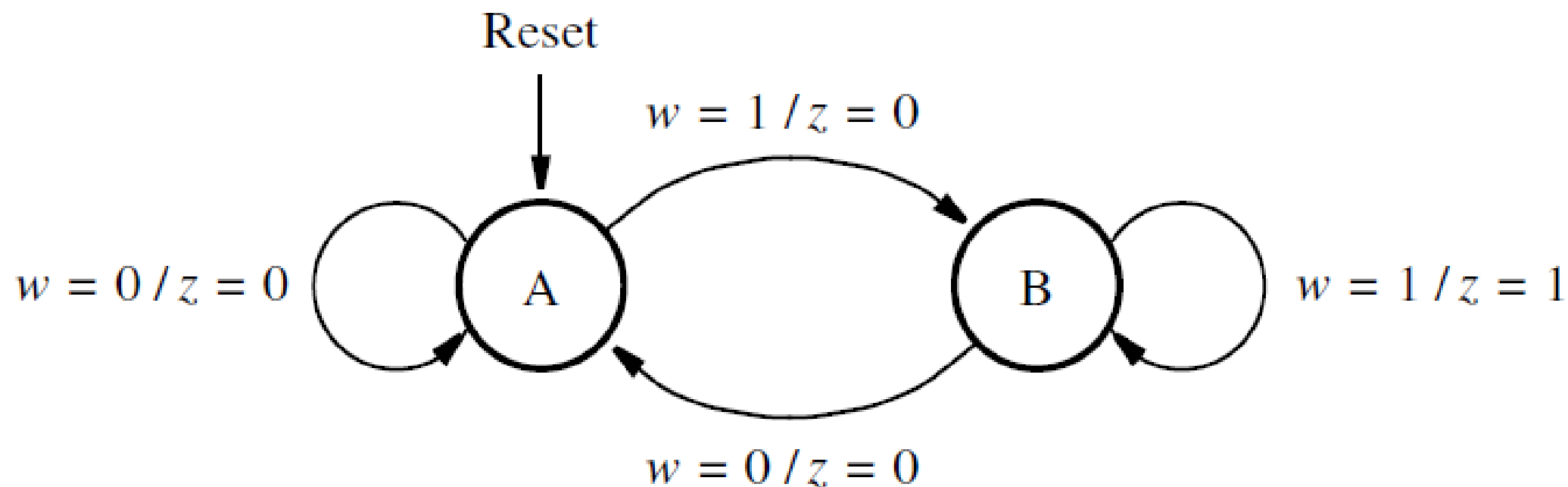
Note: This should look a lot like the counter designs done previously.

Mealy state model

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	1	0	0	1	1	0	0

Mealy state model

Clock cycle:	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8	t_9	t_{10}
w :	0	1	0	1	1	0	1	1	1	0	1
z :	0	0	0	0	1	0	0	1	1	0	0



Sequence of input –output & State Diagram

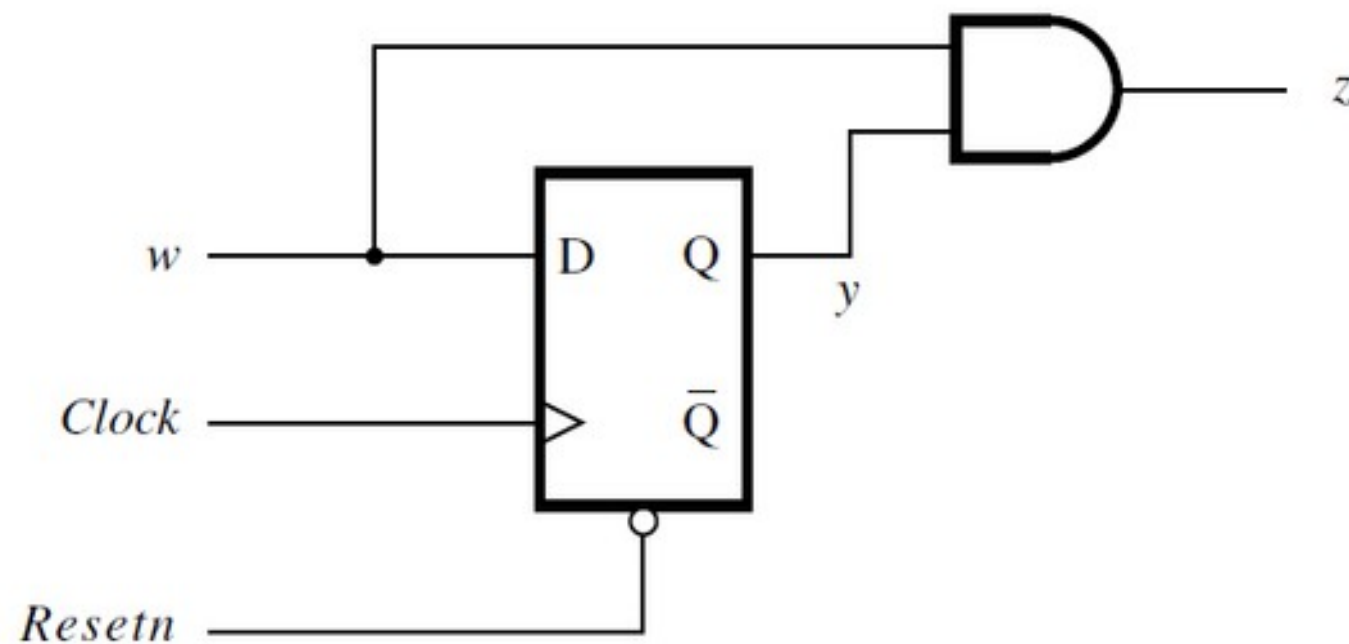
Present state	Next state		Output z	
	$w = 0$	$w = 1$	$w = 0$	$w = 1$
A	A	B	0	0
B	A	B	0	1

State TABLE for FSM

	Present state	Next state		Output	
		$w = 0$	$w = 1$	$w = 0$	$w = 1$
	y	Y	Y	z	z
A	0	0	1	0	0
B	1	0	1	0	1

State ASSIGNED TABLE for FSM

Circuit for Mealy FSM











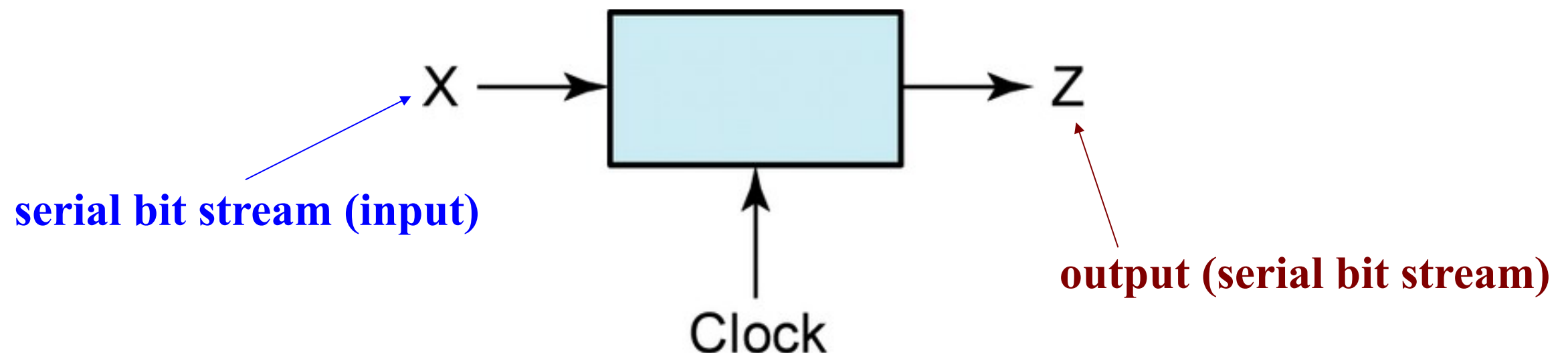
Sequential Circuit Design

- Understand specifications
- Draw state graph (to describe state machine behavior)
- Construct state table (from state graph)
- Perform state minimization (if necessary)
- Encode states (aka. state assignment)
- Create state-assigned table
- Select type of Flip-Flop to use
- Determine Flip-Flop input equations and FSM output equation(s)
- Draw logic diagram

Example: A sequence detector (Mealy)

To illustrate the design of a clocked Mealy sequential circuit, we will design a sequence detector.

The circuit is of the form:



Example: A sequence detector (Mealy)

Suppose we want to design the sequence detector so that any input sequence ending in 101 will produce an output of $Z = 1$ coincident with the last 1.

The circuit does not reset when a 1 output occurs.

A typical input sequence and the corresponding output sequence are:

$X =$ 0 0 1 1 0 1 1 0 0 1 0 1 0 1 0 0

$Z =$ 0 0 0 0 0 1 0 0 0 0 0 1 0 1 0 0

(time: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15)

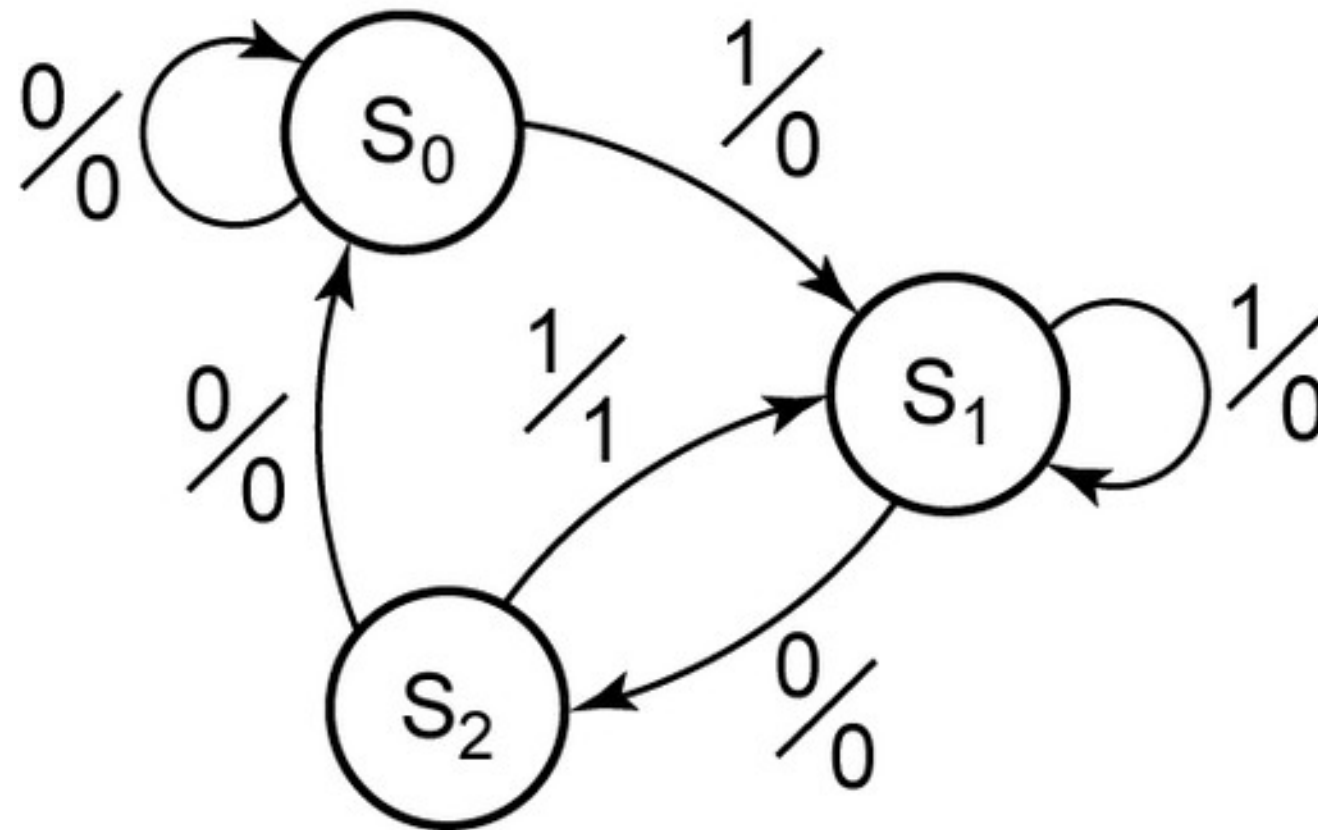
Example: A sequence detector (Mealy)

Initially, we do not know how many flip-flops will be required, so we will designate the circuit states as S_0 , S_1 , etc.

We will start with a reset state designated S_0 .

If a 0 input is received, the circuit can stay in S_0 because the input sequence we are looking for does not start with a 0.

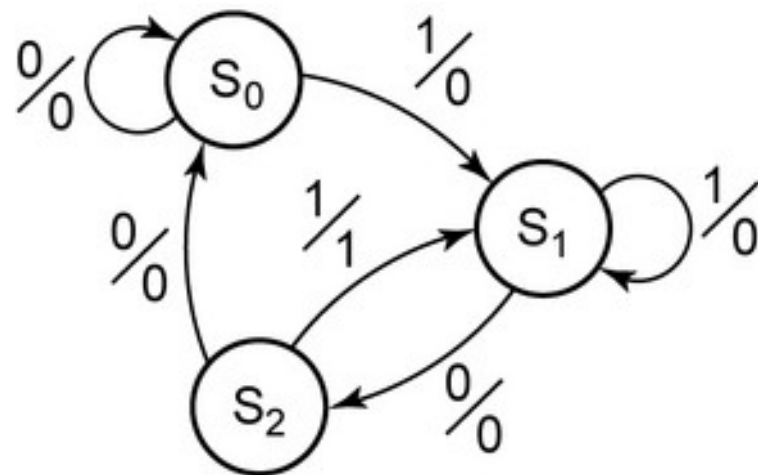
Example: A sequence detector (Mealy)



State Graph for the Mealy Machine

Example: A sequence detector (Mealy)

We can then convert our state graph to a state table:



Since there are 3 states, we only need 2 flip-flops for the circuit.

Present State	Next State		Present Output	
	$X = 0$	$X = 1$	$X = 0$	$X = 1$
S_0	S_0	S_1	0	0
S_1	S_2	S_1	0	0
S_2	S_0	S_1	0	1

Example: A sequence detector (Mealy)

And then convert our state table to a transition table:

Present State	Next State		Present Output	
	$X = 0$	$X = 1$	$X = 0$	$X = 1$
S_0	S_0	S_1	0	0
S_1	S_2	S_1	0	0
S_2	S_0	S_1	0	1

AB	A^+B^+		Z	
	$X = 0$	$X = 1$	$X = 0$	$X = 1$
00	00	01	0	0
01	10	01	0	0
10	00	01	0	1

Example: A sequence detector (Mealy)

From the transition table, we can plot the next-state maps for the flip-flops and the map for the output function Z:

		X	
		0	1
AB	00	0	0
	01	1	0
	11	X	X
	10	0	0

$$A^+ = X'B$$

		X	
		0	1
AB	00	0	1
	01	0	1
	11	X	X
	10	0	1

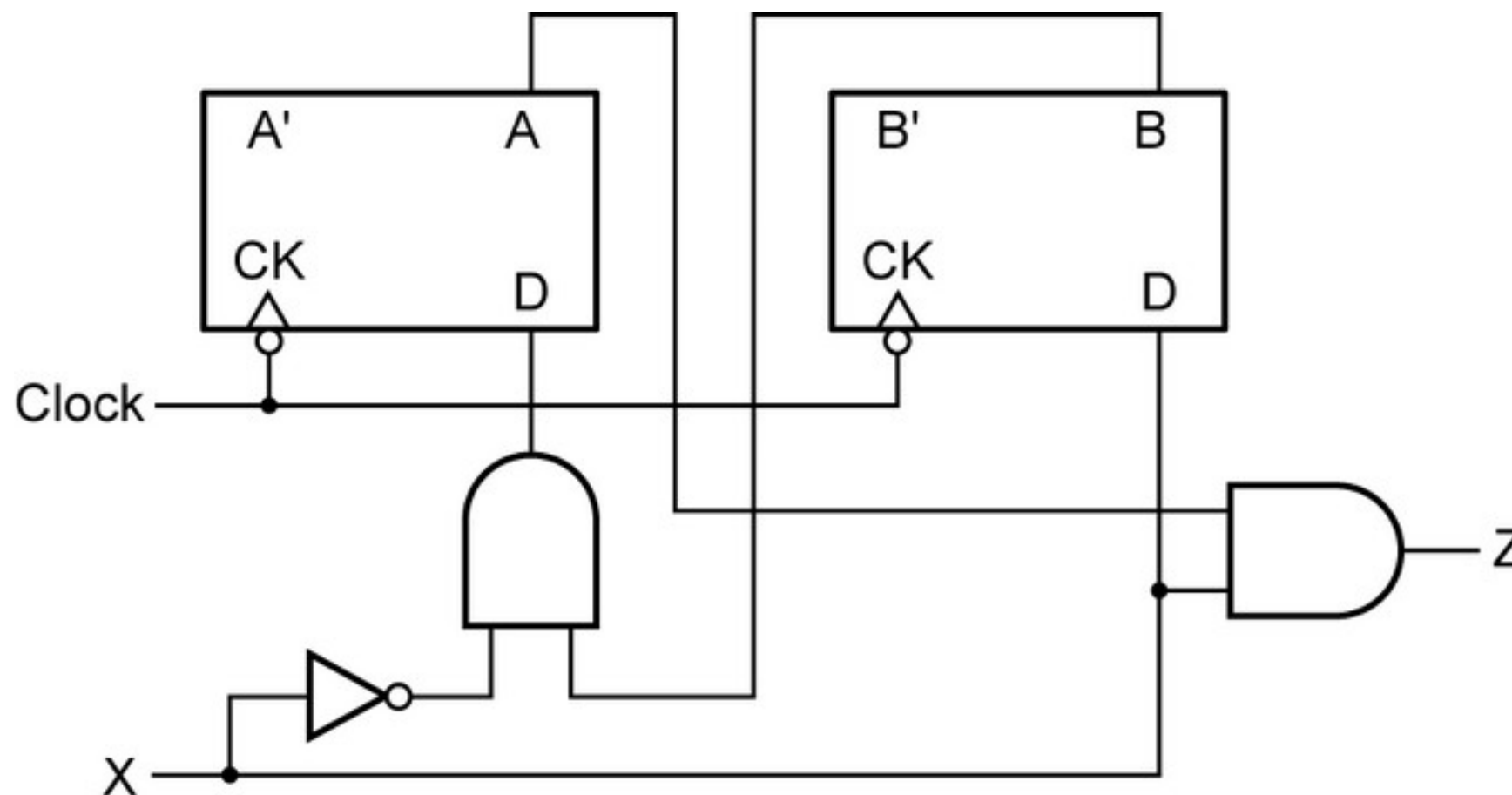
$$B^+ = X$$

		X	
		0	1
AB	00	0	0
	01	0	0
	11	X	X
	10	0	1

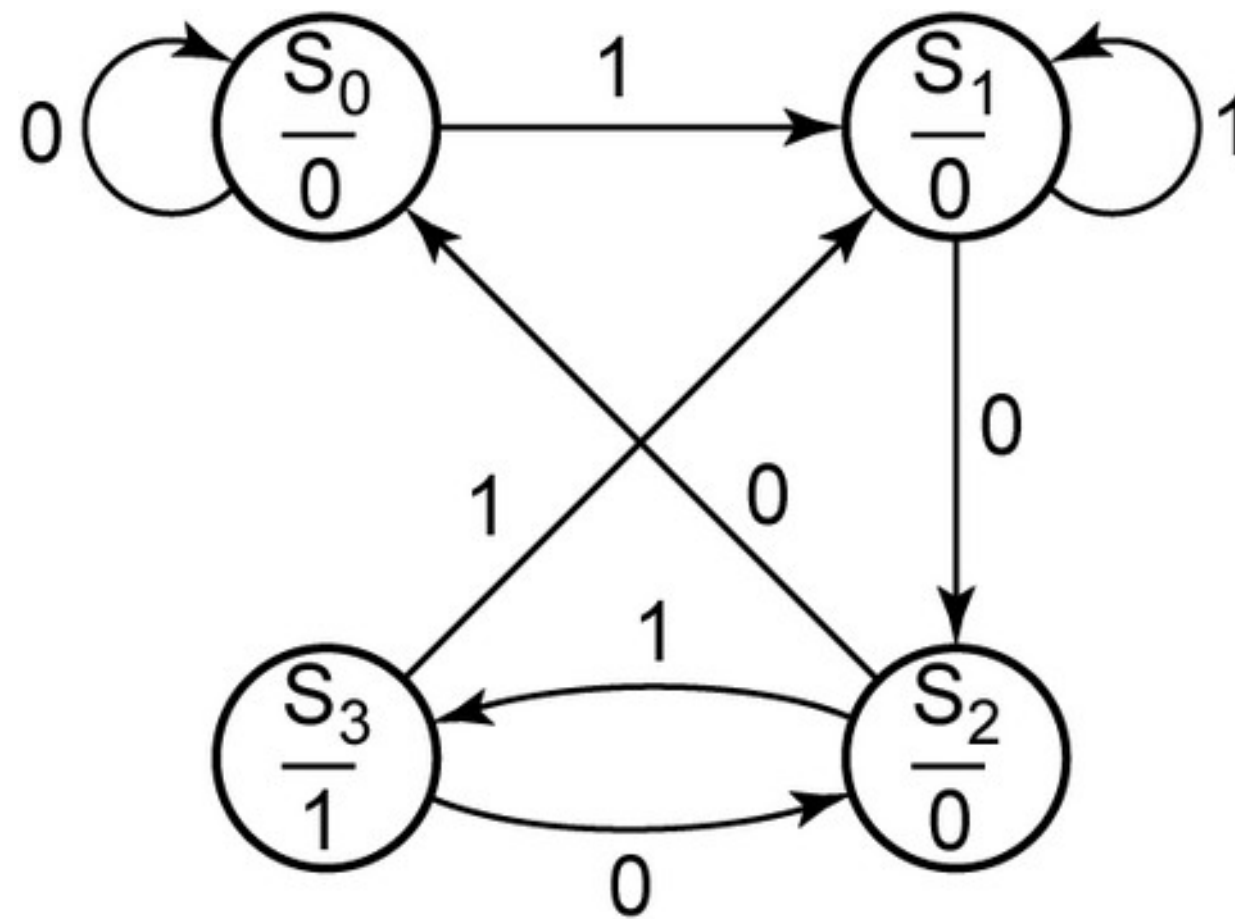
$$Z = XA$$

Example: A sequence detector (Mealy)

Using the derived equations, we can then draw the corresponding circuit diagram:



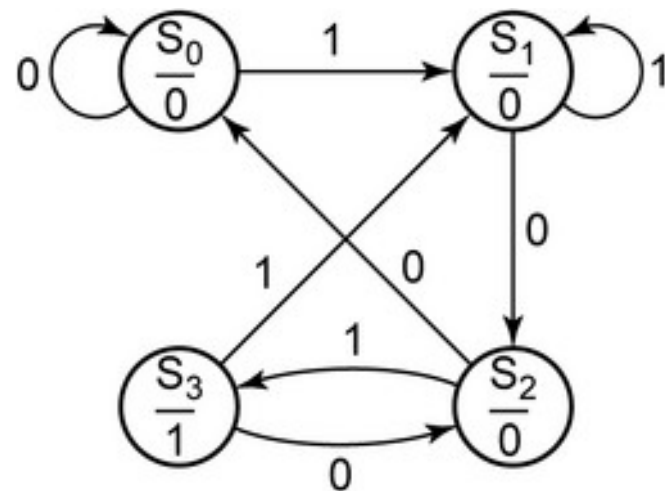
Example: A sequence detector (Moore)



State Graph for the Moore Machine

Example: A sequence detector (Moore)

As with the Mealy machine, the state and transition tables can be derived from the state graph.



Present State	Next State		Present Output(Z)
	$X = 0$	$X = 1$	
S_0	S_0	S_1	0
S_1	S_2	S_1	0
S_2	S_0	S_3	0
S_3	S_2	S_1	1

AB	A^+B^+		Z
	$X = 0$	$X = 1$	
00	00	01	0
01	11	01	0
11	00	10	0
10	11	01	1

Revision

.. For the decoder circuit shown in Figure. 2, $f_1 + f_2 =$ _____.

[CO3]

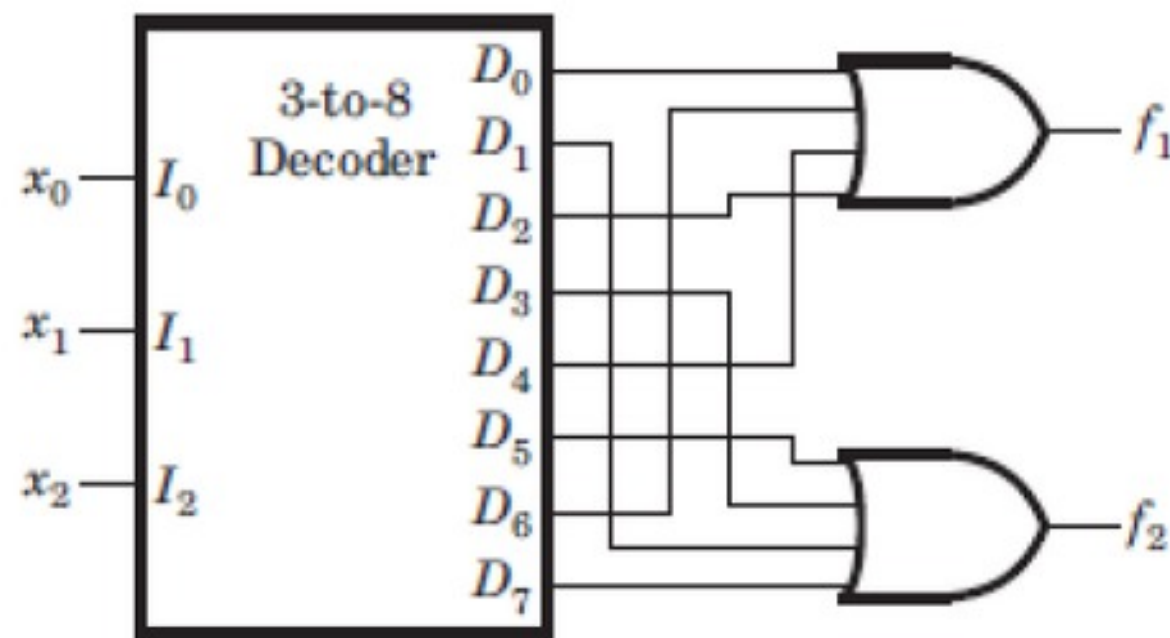


Figure.2

- a) $x_0 x_1 x_2$ b) 1 c) $x_0 + x_1 + x_2$ d) $x_0 \oplus x_1 \oplus x_2$ e) 0 f) None of the mentioned

Ans: b)

The building block shown in Figure. 3 is an active high output decoder. The output X is _____.

[CO3]

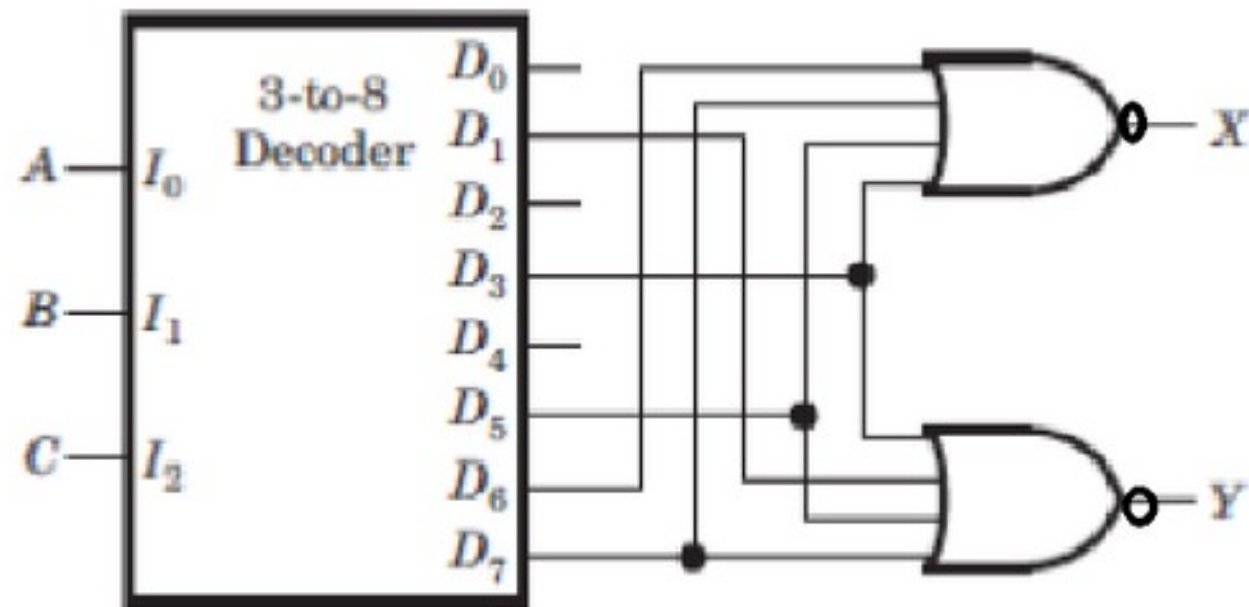


Figure.3

- a) $(A'+B').(B'+C').(C'+A')$ b) $A+B+C$ c) ABC d) $AB+BC$ e) $A+B$
f) $B+C$

Ans: a)

For the 2:1 MUX network shown in Figure. 5, $Z_1 = ?$ For $S_0 = 1$.

[CO3]

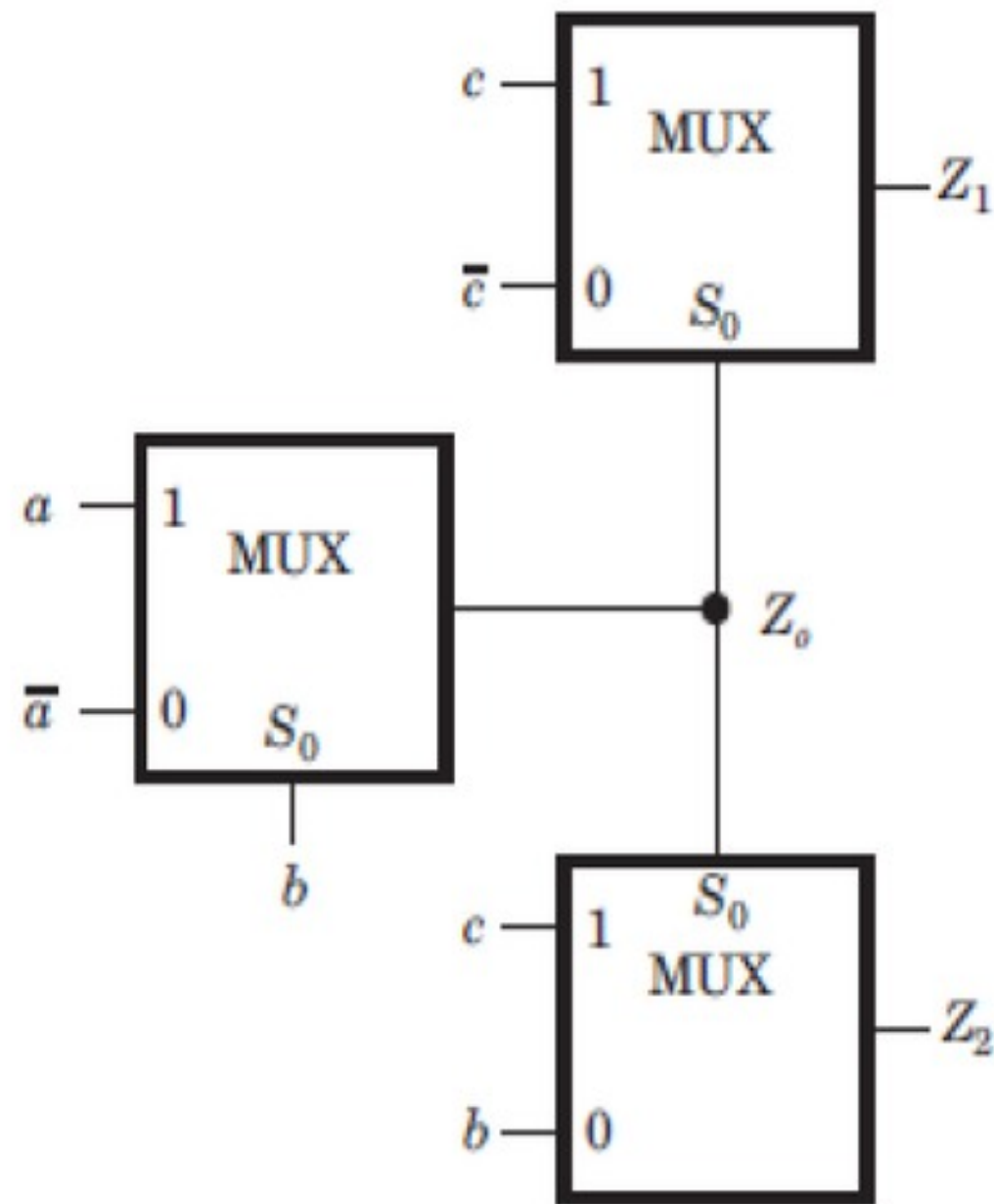


Figure. 5

a) $a + b + c$

b) $a.b.c$

c) $a \oplus b \oplus c$

d) $ab + bc + a$

e) $a' + b' + c'$

f) $c(a \odot b)$

Ans. f)

Counter Design

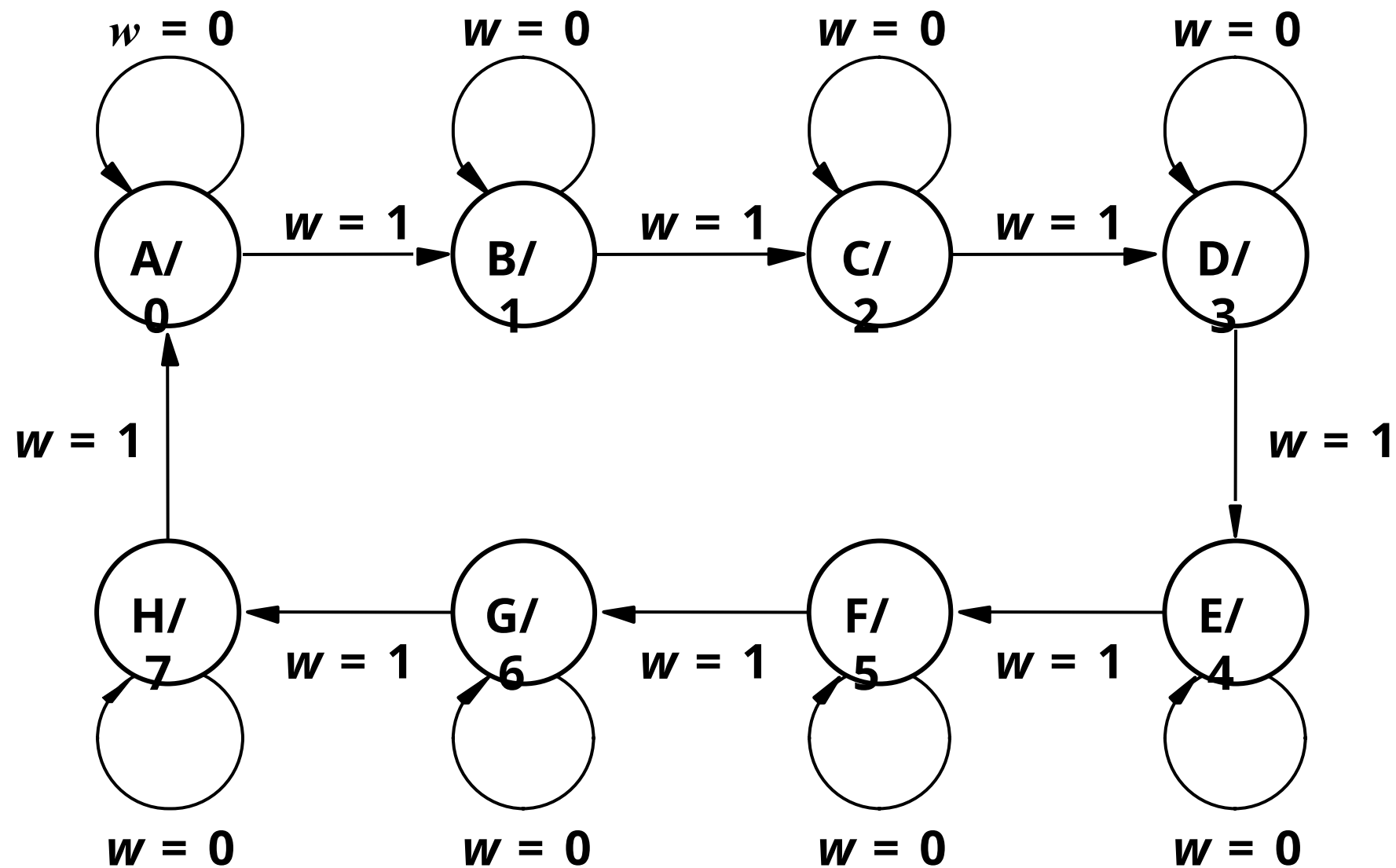


Figure 8.60. State diagram for the counter.

Present state	Next state		Output
	$w = 0$	$w = 1$	
A	A	B	0
B	B	C	1
C	C	D	2
D	D	E	3
E	E	F	4
F	F	G	5
G	G	H	6
H	H	A	7

Figure 8.61. State table for the counter.

	Present state $y_2y_1y_0$	Next state		Count $z_2z_1z_0$
		$w = 0$	$w = 1$	
		$Y_2Y_1Y_0$	$Y_2Y_1Y_0$	
A	000	000	001	000
B	001	001	010	001
C	010	010	011	010
D	011	011	100	011
E	100	100	101	100
F	101	101	110	101
G	110	110	111	110
H	111	111	000	111

Figure 8.62. State-assigned table for the counter.

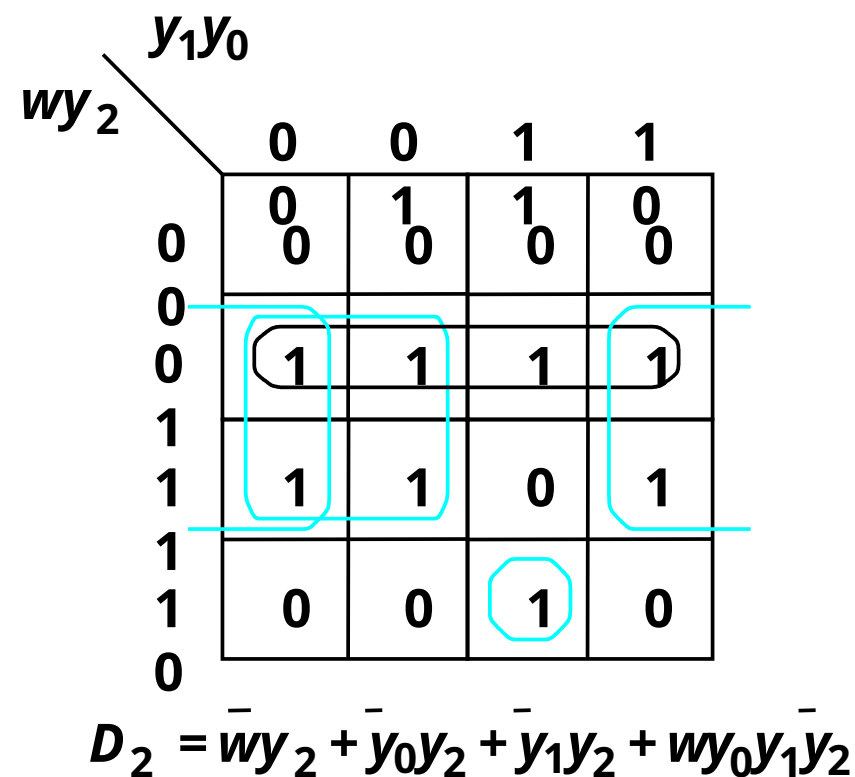
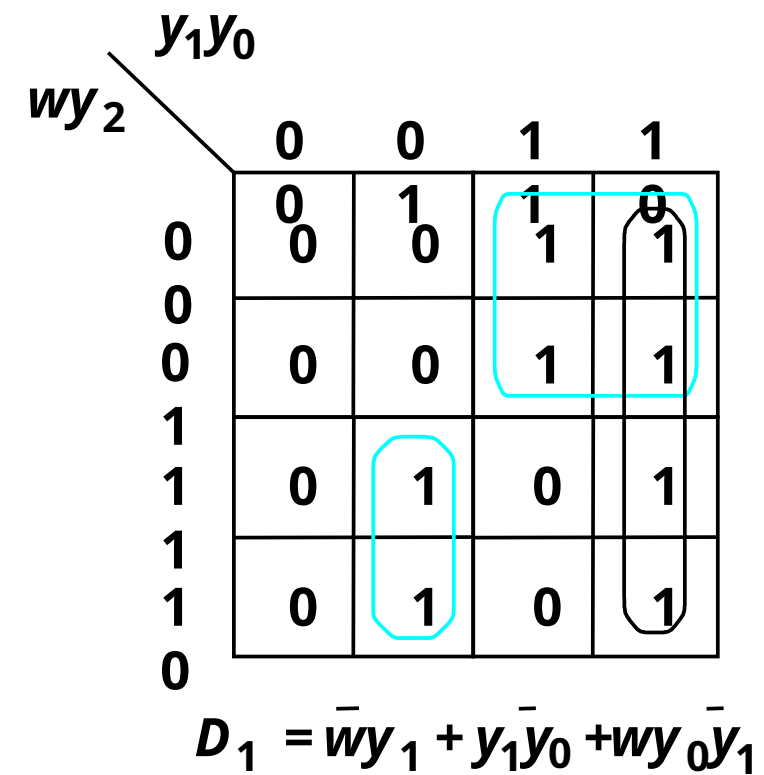
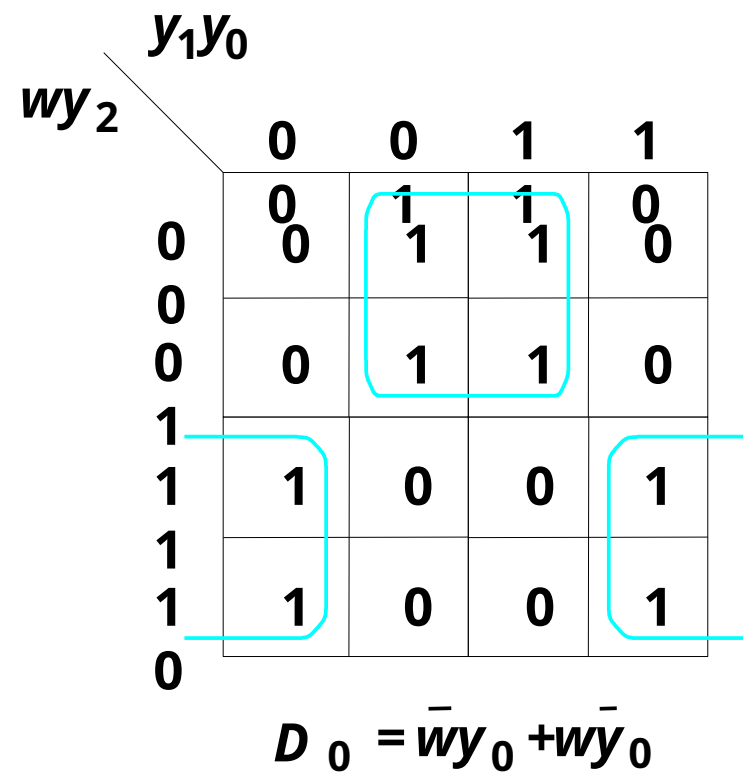
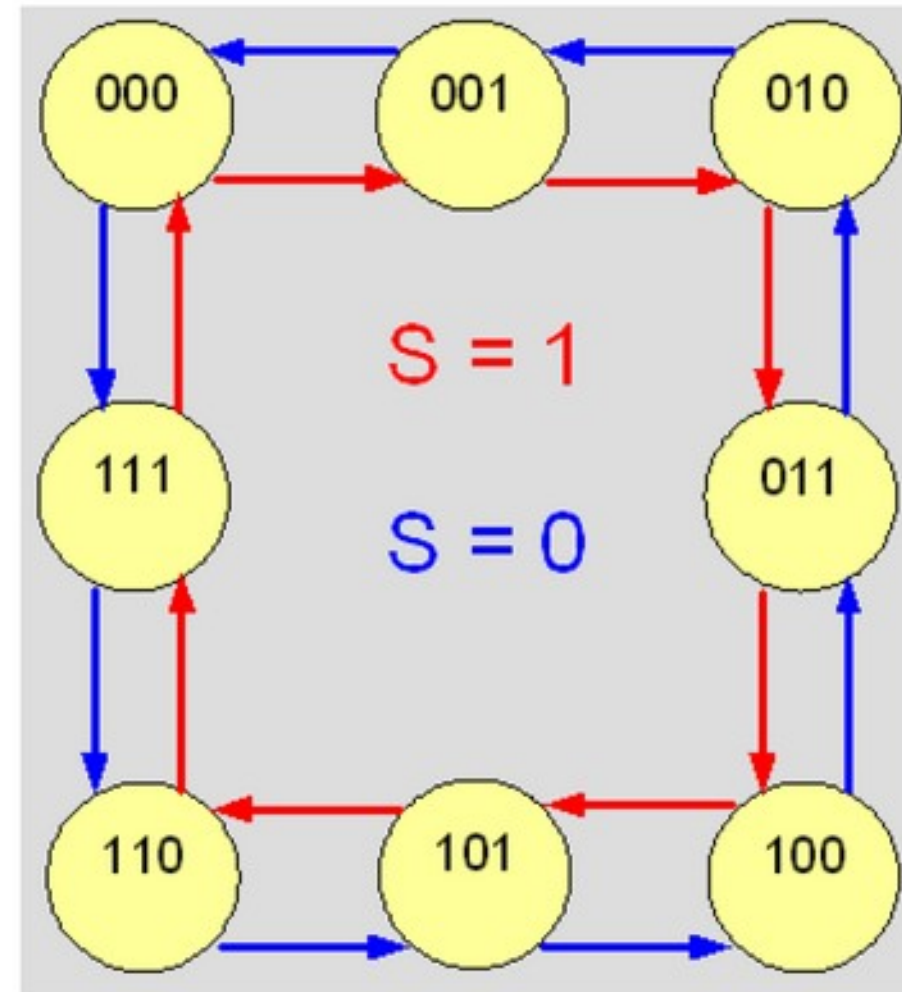


Figure 8.63. Karnaugh maps for D flip-flops for the counter.





ALL THE BEST